

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт
по лабораторной работе №1
по курсу «Технологии программирования»
на тему «Начало работы»
Вариант 5

Выполнил студент группы 23ВВВ2:
Юров Д.М.

Приняли:
к.т.н., доцент Сорокин А.А.
Патунин Д.В.

Пенза 2026

Цель работы: изучение объектно-ориентированного программирования и получение навыков написания программ, содержащих файловый ввод-вывод, контейнеры, итераторы и регулярные выражения на примере языка программирования C++.

Задание: написать программу, позволяющую пользователю извлекать из текстового документа, написанного с использованием языка разметки, определенное содержимое. Язык разметки и извлекаемое содержимое выбираются в соответствии с вариантом.

Вариант 5:

- Язык разметки – *Markdown*;
- Извлекаемое содержимое: заголовки, абзацы, упорядоченные одноуровневые списки.

Требования к программе:

- имя файла передается как аргумент командной строки;
- для создания и управления коллекциями объектов использовать контейнеры;
- для обхода контейнеров использовать итераторы;
- поиск производить посредством регулярных выражений.

Ход работы:

1. Открыл *Visual Studio Code*. Создал папку проекта, определил его структуру и инициализировал *Git*-репозиторий (рис. 1).

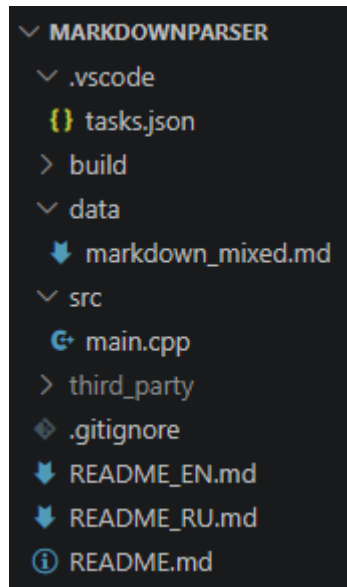


Рисунок 1 - структура проекта

Описание структуры проекта:

- каталог “*.vscode*” содержит файлы с настройками *Visual Studio Code*;
- каталог “*build*” содержит скомпилированные файлы проекта;
- каталог “*data*” содержит файлы с данными для обработки программой;
- каталог “*src*” содержит файлы исходного кода программы;
- каталог “*third_party*” содержит подключаемые сторонние библиотеки, используемые программой;
- файл “*tasks.json*” содержит задачу сборки проекта для компилятора *G++*;
- файл “*markdown_mixed.md*” содержит *Markdown*-разметку;
- файл “*main.cpp*” содержит исходный код программы;
- файл “*.gitignore*” содержит указания игнорирования файлов для *Git*-репозитория;
- файл “*README_EN.md*” содержит информацию о проекте на английском языке;

- файл “*README_RU.md*” содержит информацию о проекте на русском языке;
- файл “*README.md*” содержит краткую информацию о проекте и ссылки на файлы с локализованными версиями файлов информации о проекте – отображается на странице *Github* проекта.

2. Выбрал стороннюю библиотеку для парсинга аргументов командной строки при вызове приложения – *CL111*. Склонировал *Git*-репозиторий проекта в каталог “*third_party*” (рис. 2). Добавил в файл “*.gitignore*” указание на игнорирование каталога “*third_party*” (см. Приложение 4).

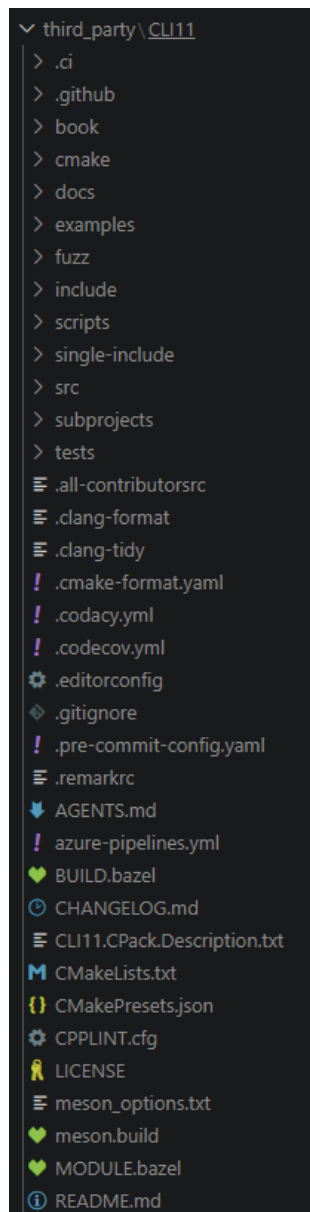
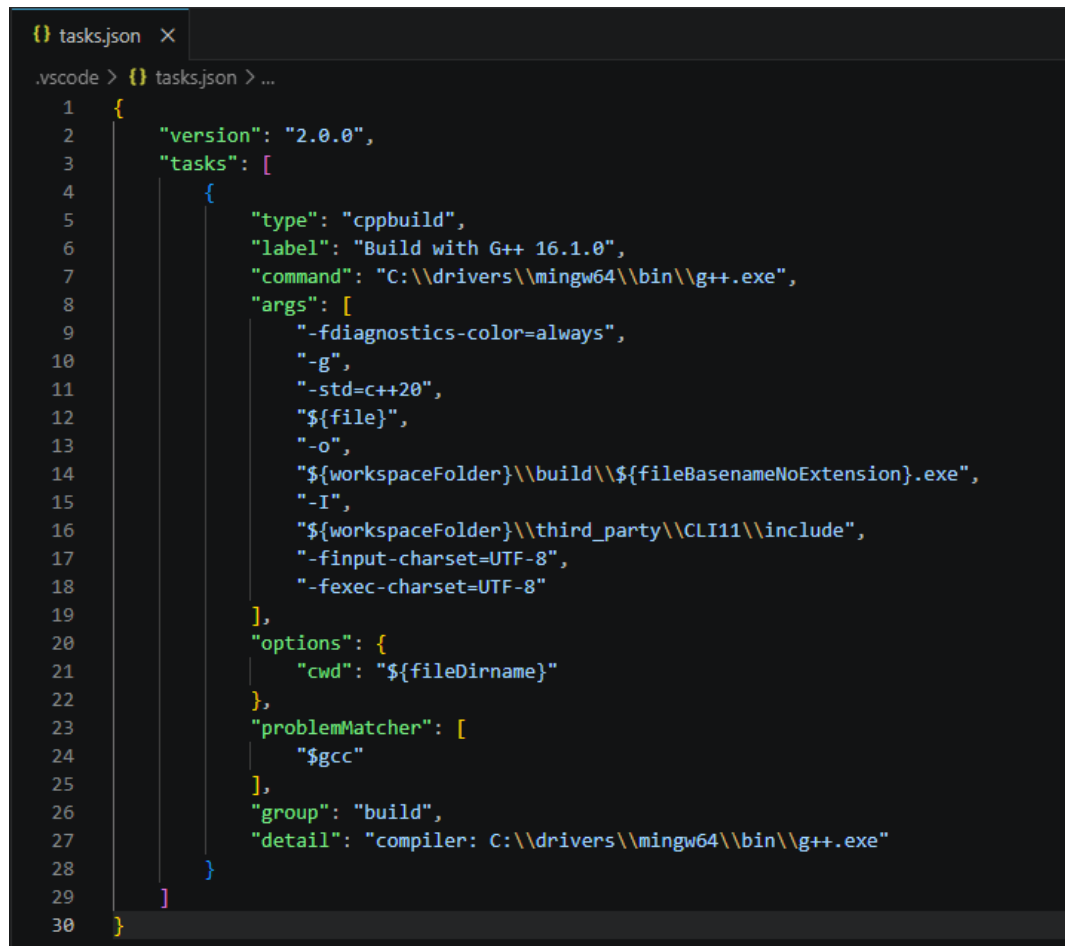


Рисунок 2 - каталог со сторонней библиотекой *CL111*

3. Добавил в файл *“tasks.json”* задачу сборки проекта для компилятора *G++*: указал использование *C++20*; команду вызова компилятора; используемую кодировку проекта – *UTF-8*; каталог, название файла и название результирующего файла для компиляции; каталог подключения сторонней библиотеки *CLI11* и другие флаги компиляции (рис. 3, см. Приложение 1).



```
1  {
2      "version": "2.0.0",
3      "tasks": [
4          {
5              "type": "cppbuild",
6              "label": "Build with G++ 16.1.0",
7              "command": "C:\\drivers\\mingw64\\bin\\g++.exe",
8              "args": [
9                  "-fdiagnostics-color=always",
10                 "-g",
11                 "-std=c++20",
12                 "${file}",
13                 "-o",
14                 "${workspaceFolder}\\build\\${fileBasenameNoExtension}.exe",
15                 "-I",
16                 "${workspaceFolder}\\third_party\\CLI11\\include",
17                 "-finput-charset=UTF-8",
18                 "-fexec-charset=UTF-8"
19             ],
20              "options": {
21                  "cwd": "${fileDirname}"
22              },
23              "problemMatcher": [
24                  "$gcc"
25              ],
26              "group": "build",
27              "detail": "compiler: C:\\drivers\\mingw64\\bin\\g++.exe"
28          }
29      ]
30  }
```

Рисунок 3 - содержимое файла *“tasks.json”*

4. С помощью ИИ-ассистента *ChatGPT* подготовил содержимое файлов *“markdown_mixed.md”* (см. Приложение 2), *“README_EN.md”* (см. Приложение 5), *“README_RU.md”* (см. Приложение 6), *“README.md”* (см. Приложение 7).

5. Следуя принципам ООП, в файле *“main.cpp”* написал исходный код программы (см. Приложение 3):

- подключил необходимые библиотеки и пространства имён;

- создал структуру *ParseResults* для хранения результатов парсинга файла с *Markdown*-разметкой;
- создал класс *CmdArgsParser*, содержащий поля и методы, необходимые для парсинга аргументов командной строки при вызове программы;
- создал класс *MarkdownParser*, содержащий поля и методы, необходимые для парсинга файла с *Markdown*-разметкой;
- создал главную функцию *main*.

6. С помощью неоднократного тестирования программы добавил обработку большинства ошибок, исправил недочёты исходного кода и проверил работоспособность финальной версии программы в нескольких сценариях.

Описание исходного кода программы.

В программе используются следующие подключаемые библиотеки:

- библиотека “*windows.h*” используется для настройки кодировки консоли *Windows* с помощью функций *SetConsoleCP* и *SetConsoleOutputCP*;
- библиотека “*iostream*” используется для ввода-вывода данных через стандартные потоки *cout* и *cerr*;
- библиотека “*fstream*” используется для файлового ввода, то есть для открытия и чтения *Markdown*-файла;
- библиотека “*string*” используется для работы со строковыми данными;
- библиотека “*vector*” используется для хранения коллекций строк и результатов парсинга;
- библиотека “*regex*” используется для поиска элементов *Markdown*-разметки с помощью регулярных выражений;
- библиотека “*locale*” используется для настройки локализации программы;

- библиотека “*iterator*” используется для работы с итераторами контейнеров;
- библиотека “*stdexcept*” используется для работы со стандартными исключениями.

В программе используется сторонняя библиотека *CLII*, подключаемая с помощью заголовочного файла “*CLI/CLI.hpp*”. Она применяется для обработки аргументов командной строки, проверки обязательных параметров и ограничения допустимых значений режима работы программы.

В программе используются пространства имён “*std*” и “*CLI*”. Они применяются для удобства работы с кодом и позволяют обращаться к элементам стандартной библиотеки C++ и библиотеки *CLII* без постоянного указания префиксов.

В программе реализована структура *ParseResults* для хранения результатов парсинга файла с *Markdown*-разметкой. Структура содержит следующие поля:

- поле `vector<string> headers` является контейнером полученных заголовков;
- поле `vector<string> paragraphs` является контейнером полученных абзацев;
- поле `vector<string> o_lists` является контейнером полученных одноуровневых упорядоченных списков.

В программе реализован класс *CmdArgsParser* для парсинга аргументов командной строки при вызове программы. Класс содержит следующие приватные поля:

- поле `int modeInt` хранит целочисленное представление режима работы программы. Значение 1 соответствует режиму *all*, значение 2 — режиму *headers*, значение 3 — режиму *paragraphs*, значение 4 — режиму *o_lists*;

- поле *string fileName* хранит путь к файлу, который должен быть обработан программой.

Класс *CmdArgsParser* содержит следующие методы:

- приватный метод *int makeModeInt(string mode)* принимает строковое значение режима работы программы и преобразует его в целочисленное значение. Это нужно для дальнейшего удобного выбора режима обработки с помощью оператора *switch*;
- публичный метод *bool parseArgs(int argc, char** argv)* принимает аргументы командной строки, создаёт объект конфигурации *CLIII*, добавляет обязательные параметры *--mode* и *--file*, проверяет корректность выбранного режима и сохраняет полученные значения во внутренние поля класса. Параметр *--mode* может принимать значения *all*, *headers*, *paragraphs* или *o_lists*. Параметр *--file* содержит путь к *Markdown*-файлу;
- публичный метод *string getFileName()* возвращает путь к файлу, полученный из аргументов командной строки;
- публичный метод *int getMode()* возвращает целочисленное значение выбранного режима работы программы.

В программе реализован класс *MarkdownParser*, содержащий регулярные выражения и методы, необходимые для чтения файла, поиска заголовков, поиска абзацев, поиска упорядоченных одноуровневых списков и вывода результата.

В приватной части класса *MarkdownParser* объявлены два регулярных выражения:

- регулярное выражение *re_header* используется для поиска *Markdown*-заголовков от первого до шестого уровня. Оно распознаёт строки, начинающиеся с одного или нескольких символов “#”, после которых находится пробел и текст заголовка;

- регулярное выражение *re_olist* используется для поиска элементов упорядоченного списка. Оно распознаёт строки, начинающиеся с числа, точки, пробела и текста элемента списка.

Класс *MarkdownParser* содержит следующие приватные методы:

- метод *vector read_lines(const string& filename)* открывает файл, путь к которому передан в параметре *filename*, считывает его построчно и сохраняет все строки в контейнер *vector*. Если файл не удалось открыть, метод генерирует исключение *runtime_error*;
- метод *void parseHeaders(const vector& lines, vector& result)* проходит по всем строкам файла и с помощью регулярного выражения *re_header* находит *Markdown*-заголовки. Найденный текст заголовка без символов “#” добавляется в контейнер *result*;
- метод *void parseParagraphs(const vector& lines, vector& result)* проходит по всем строкам файла и добавляет в контейнер *result* строки, которые не являются пустыми, не являются заголовками и не являются элементами упорядоченного списка. Таким образом программа выделяет обычные текстовые строки как абзацы;
- метод *void parseOLists(const vector& lines, vector& result)* проходит по строкам файла с помощью итератора, находит последовательности строк, похожих на элементы упорядоченного списка, и временно сохраняет их в контейнер *currentList*. После завершения очередной последовательности метод проверяет список на корректность и, если список является одноуровневым упорядоченным списком, добавляет его элементы в результирующий контейнер;
- метод *bool isOrderedOneLevelList(const vector& lines)* проверяет корректность найденного упорядоченного списка. Метод проверяет, что список не пустой, что каждая строка соответствует шаблону элемента упорядоченного списка, а нумерация элементов идёт последовательно, начиная с единицы.

В публичной части класса *MarkdownParser* реализованы следующие методы:

- метод *void print(int mode, ParseResults& res)* выводит результаты парсинга в консоль. В зависимости от выбранного режима метод выводит либо все найденные элементы, либо только заголовки, либо только абзацы, либо только упорядоченные одноуровневые списки;
- метод *ParseResults parse(const int& mode, const string& fileName)* выполняет основную обработку файла. Сначала метод вызывает *read_lines* и получает все строки исходного файла. Затем, в зависимости от выбранного режима, вызывает методы *parseHeaders*, *parseParagraphs* и *parseOLists*. Результаты сохраняются в структуру *ParseResults*, которая затем возвращается из метода.

Главная функция *main* является точкой входа в программу. В начале функции настраивается кодировка консоли Windows на *UTF-8* с помощью функций *SetConsoleCP* и *SetConsoleOutputCP*, а также задаётся локализация программы с помощью *setlocale*. Затем создаётся объект класса *CmdArgsParser*, который обрабатывает аргументы командной строки. Если аргументы командной строки указаны некорректно, программа завершает выполнение с кодом 99.

После успешной обработки аргументов командной строки создаётся объект класса *MarkdownParser*. Далее в блоке *try* вызывается метод *parse*, который выполняет чтение и разбор *Markdown*-файла. Полученные результаты сохраняются в структуру *ParseResults*. После этого вызывается метод *print*, который выводит найденные элементы в консоль в соответствии с выбранным режимом работы программы.

Для обработки ошибок в функции *main* используется блок *catch*. Если во время выполнения программы возникает исключительная ситуация, например файл не удалось открыть или был передан неверный режим работы, сообщение об ошибке выводится в поток *cerr*.

Тестирование программы.

Тестирование автоматически собираемой help-страницы (рис. 4).

```
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --help
parser v1.0 by purple

D:\PSU\5_term\TP\res\lab1\MarkdownParser\build\main.exe [OPTIONS]

OPTIONS:
  -h,  --help          Print this help message and exit
      --mode TEXT:{all,headers,paragraphs,o_lists} REQUIRED
                        Режимы:
                        | all - все режимы в одном |
                        | headers - выводит только заголовки |
                        | paragraphs - выводит только параграфы |
                        | o_lists - парсит только упорядоченные
                        | одноуровневые списки |
      --file TEXT REQUIRED
                        Путь к файлу для парсинга
```

Рисунок 4 - тестирование *help*-страницы

Тестирование запуска с разными параметрами (рис. 5-6).

```
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --mode all --file "./data/markdown_mixed.md"
заголовки:
Пример файла с элементами Markdown
Введение
Основные принципы
Практическое применение
Заключение

параграфы:
Это демонстрационный файл, где заголовки, абзацы и упорядоченные списки перемешаны с обычным текстом.
---
Markdown — это лёгкий язык разметки, который позволяет оформлять текст без использования сложных инструментов.
После списка может идти обычный текст: например, вы можете продолжить абзац, не прерывая структуру документа.
---
В Markdown можно свободно чередовать текст и списки.
Вот пример последовательности элементов:
Обычный текст, который вводит читателя в тему.
А дальше снова идёт обычный абзац, без каких-либо специальных элементов.
Он может включать жирное или курсивное выделение.
---
Markdown часто используется для написания:
После списка можно добавить заключительный абзац, который подводит итог раздела.
---
Markdown даёт возможность создавать структурированный и читаемый текст, сохраняя при этом его простоту.
Вы можете свободно комбинировать заголовки, абзацы и списки, чтобы текст выглядел логично и аккуратно.

упорядоченные одноуровневые списки:
1. Он прост в использовании.
2. Поддерживается на многих платформах.
3. Удобен для документирования и заметок.
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> █
```

Рисунок 5 - тестирование при разных значениях флага *mode* 1

```

PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --mode headers --file "./data/markdown_mixed.md"
заголовки:
Пример файла с элементами Markdown
Введение
Основные принципы
Практическое применение
Заключение
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --mode paragraphs --file "./data/markdown_mixed.md"
параграфы:
Это демонстрационный файл, где заголовки, абзацы и упорядоченные списки перемешаны с обычным текстом.
---
Markdown — это лёгкий язык разметки, который позволяет оформлять текст без использования сложных инструментов.
После списка может идти обычный текст: например, вы можете продолжить абзац, не прерывая структуру документа.
---
В Markdown можно свободно чередовать текст и списки.
Вот пример последовательности элементов:
Обычный текст, который вводит читателя в тему.
А дальше снова идёт обычный абзац, без каких-либо специальных элементов.
Он может включать жирное или курсивное выделение.
---
Markdown часто используется для написания:
После списка можно добавить заключительный абзац, который подводит итог раздела.
---
Markdown даёт возможность создавать структурированный и читаемый текст, сохраняя при этом его простоту.
Вы можете свободно комбинировать заголовки, абзацы и списки, чтобы текст выглядел логично и аккуратно.
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --mode o_lists --file "./data/markdown_mixed.md"
упорядоченные одноуровневые списки:
1. Он прост в использовании.
2. Поддерживается на многих платформах.
3. Удобен для документирования и заметок.

```

Рисунок 6 - тестирование при разных значениях флага *mode* 2

Тестирование при отсутствии одного из флагов (рис. 7).

```

PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --mode all
--file is required
Run with --help for more information.
Не удалось открыть файл:
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe --file "./data/markdown_mixed.md"
--mode is required
Run with --help for more information.
Не удалось открыть файл:
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser> ./build/main.exe
--mode is required
Run with --help for more information.
Не удалось открыть файл:
PS D:\PSU\5_term\TP\res\lab1\MarkdownParser>

```

Рисунок 7 - тестирование при отсутствии флагов

Вывод: в ходе выполнения лабораторной работы была написана программа, позволяющая пользователю извлекать заголовки, абзацы и упорядоченные одноуровневые списки из текстового документа, написанного с использованием языка *Markdown*-разметки. Были получены навыки написания программ, содержащих файловый ввод-вывод, контейнеры, итераторы и регулярные выражения на примере языка C++.

Приложение 1 – содержимое файла “tasks.json”

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "cppbuild",
      "label": "Build with G++ 16.1.0",
      "command": "C:\\drivers\\mingw64\\bin\\g++.exe",
      "args": [
        "-fdiagnostics-color=always",
        "-g",
        "-std=c++20",
        "${file}",
        "-o",
        "${workspaceFolder}\\build\\${fileBasenameNoExtension}.exe",
        "-I",
        "${workspaceFolder}\\third_party\\CLI11\\include",
        "-finput-charset=UTF-8",
        "-fexec-charset=UTF-8"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": [
        "$gcc"
      ],
      "group": "build",
      "detail": "compiler: C:\\drivers\\mingw64\\bin\\g++.exe"
    }
  ]
}
```

Приложение 2 – содержимое файла “*markdown_mixed.md*”

Пример файла с элементами Markdown

Это демонстрационный файл, где ****заголовки****, *_абзацы_* и ****упорядоченные списки**** перемешаны с обычным текстом.

Введение

Markdown – это лёгкий язык разметки, который позволяет оформлять текст без использования сложных инструментов.

1. Он прост в использовании.
2. Поддерживается на многих платформах.
3. Удобен для документирования и заметок.

После списка может идти обычный текст: например, вы можете продолжить абзац, не прерывая структуру документа.

Основные принципы

В Markdown можно свободно чередовать текст и списки. Вот пример последовательности элементов:

Обычный текст, который вводит читателя в тему.

1. Первый пункт списка.
2. Второй пункт списка, который может быть длиннее и занимать несколько строк, если требуется пояснение.
5. Третий пункт.

А дальше снова идёт обычный абзац, без каких-либо специальных элементов. Он может включать ****жирное**** или **курсивное** выделение.

Практическое применение

Markdown часто используется для написания:

1. README-файлов; первый пункт
3. Документации к проектам; второй пункт
2. Учебных материалов и конспектов. третий пункт

После списка можно добавить заключительный абзац, который подводит итог раздела.

Заключение

Markdown даёт возможность создавать структурированный и читаемый текст, сохраняя при этом его простоту.

Вы можете свободно комбинировать ****заголовки****, **абзацы** и ****списки****, чтобы текст выглядел логично и аккуратно.

Приложение 3 – содержимое файла “*main.cpp*”

```
#include <windows.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <regex>
#include <locale>
#include <iterator>
#include <stdexcept>

#include <CLI/CLI.hpp>

using namespace std;
using namespace CLI;

struct ParseResults
{
    vector<string> headers;
    vector<string> paragraphs;
    vector<string> o_lists;
};

class CmdArgsParser
{
private:
    int modeInt = 0; //1 - all, 2 - headers, 3 - paragraphs, 4 - o_lists
    string fileName;

    int makeModeInt(string mode)
    {
        if (mode == "all")
        {
            return 1;
        }
        else if (mode == "headers")
        {
            return 2;
        }
        else if (mode == "paragraphs")
        {
            return 3;
        }
        else if (mode == "o_lists")
        {
            return 4;
        }
    }
}
```

```

        else return 0;
    }

public:
    bool parseArgs(int argc, char** argv)
    {
        string modeStr;
        App config{ "parser v1.0 by purple" };

        config.add_option("--mode", modeStr,
            "Режимы: \n | all - все режимы в одном |\n"
            " | headers - выводит только заголовки |\n"
            " | paragraphs - выводит только параграфы |\n"
            " | o_lists - парсит только упорядоченные одноуровневые списки |")
            ->required()
            ->check(IsMember({ "all", "headers", "paragraphs", "o_lists" }));

        config.add_option("--file", fileName, "Путь к файлу для парсинга")
            ->required();

        try
        {
            config.parse(argc, argv);

            modeInt = makeModeInt(modeStr);
            if (!modeInt)
            {
                return false;
            }

            return true;
        }
        catch (const ParseError& e)
        {
            config.exit(e);
            return false;
        }
    }

    string getFileName()
    {
        return fileName;
    }

    int getMode()
    {
        return modeInt;
    }
};

class MarkdownParser {

```

```

private:
    regex re_header = regex(R"(\s{0,3}(\#{1,6})\s+(.*)$)");
    regex re_olist = regex(R"(\s*(\d+)\.\s+(.*)$)");

    vector<string> read_lines(const string& filename)
    {
        ifstream file(filename);
        if (!file.is_open())
        {
            throw runtime_error("Не удалось открыть файл: " + filename);
        }

        vector<string> lines;
        string line;

        while (getline(file, line))
            lines.push_back(line);

        return lines;
    }

    void parseHeaders(const vector<string>& lines, vector<string>& result)
    {
        for (const auto& line : lines) {
            smatch m;
            if (regex_match(line, m, re_header)) {
                result.push_back(m[2].str());
            }
        }
    }

    void parseParagraphs(const vector<string>& lines, vector<string>& result)
    {
        for (const auto& line : lines) {
            if (line.empty()) continue;

            if (regex_match(line, re_header)) continue;
            if (regex_match(line, re_olist)) continue;

            if (!line.empty())
                result.push_back(line);
        }
    }

    void parseOLists(const vector<string>& lines, vector<string>& result)
    {
        vector<string> currentList;

        for (auto it = lines.cbegin(); it != lines.cend(); ++it)
        {
            const string& line = *it;

```

```

        if (regex_match(line, re_olist))
        {
            currentList.push_back(line);
        }
        else
        {
            if (!currentList.empty())
            {
                if (isOrderedOneLevelList(currentList))
                {
                    for (const auto& itemLine : currentList)
                    {
                        result.push_back(itemLine);
                    }
                    currentList.clear();
                }
            }
        }
    }

    if (!currentList.empty())
    {
        if (isOrderedOneLevelList(currentList))
        {
            for (const auto& itemLine : currentList)
            {
                result.push_back(itemLine);
            }
            currentList.clear();
        }
    }
}

bool isOrderedOneLevelList(const vector<string>& lines)
{
    if (lines.empty())
        return false;

    int expected = 1;

    for (const auto& line : lines)
    {
        smatch m;
        if (!regex_match(line, m, re_olist))
            return false;

        int num = stoi(m[1].str());

        if (num != expected)
            return false;
    }
}

```

```

        expected++;
    }
    return true;
}

public:
void print(int mode, struct ParseResults& res)
{
    switch (mode)
    {
        case 1:
        {
            cout << "заголовки: " << endl;
            for (const auto& header : res.headers)
            {
                cout << header << endl;
            }

            cout << endl << endl << endl << "параграфы: " << endl;
            for (const auto& paragraph : res.paragraphs)
            {
                cout << paragraph << endl;
            }

            cout << endl << endl << endl << "упорядоченные одноуровневые
списки: " << endl;
            for (const auto& o_list : res.o_lists)
            {
                cout << o_list << endl;
            }
            break;
        }
        case 2:
        {
            cout << "заголовки: " << endl;
            for (const auto& header : res.headers)
            {
                cout << header << endl;
            }
            break;
        }
        case 3:
        {
            cout << "параграфы: " << endl;
            for (const auto& paragraph : res.paragraphs)
            {
                cout << paragraph << endl;
            }
            break;
        }
        case 4:

```

```

        {
            cout << "упорядоченные одноуровневые списки: " << endl;
            for (const auto& o_list : res.o_lists)
            {
                cout << o_list << endl;
            }
            break;
        }
    }
}

struct ParseResults parse(const int& mode, const string& fileName)
{
    struct ParseResults res;
    vector<string> lines = read_lines(fileName);

    switch (mode)
    {
    case 1:
    {
        parseHeaders(lines, res.headers);
        parseParagraphs(lines, res.paragraphs);
        parseOLists(lines, res.o_lists);
        break;
    }
    case 2:
    {
        parseHeaders(lines, res.headers);
        break;
    }
    case 3:
    {
        parseParagraphs(lines, res.paragraphs);
        break;
    }
    case 4:
    {
        parseOLists(lines, res.o_lists);
        break;
    }
    default:
    {
        throw runtime_error("Неверный режим: " + to_string(mode));
    }
    }
    return res;
}

};

int main(int argc, char** argv)
{

```

```
//setlocale(LC_ALL, "Russian");

SetConsoleCP(CP_UTF8);
SetConsoleOutputCP(CP_UTF8);
setlocale(LC_ALL, ".UTF-8");

CmdArgsParser config;
if(!config.parseArgs(argc, argv)) return 99;

MarkdownParser Mark;

try
{
    struct ParseResults Results = Mark.parse(config.getMode(),
config.getFileName());

    Mark.print(config.getMode(), Results);
}
catch (const exception& ex)
{
    cerr << ex.what() << "\n";
}

return 0;
}
```

Приложение 4 – содержимое файла “.gitignore”

third_party/

Приложение 5 – содержимое файла “*README_EN.md*”

This README generated with the assistance of AI.

MarkdownParser

Console application written in C++ for parsing Markdown files and extracting selected markup elements.

The program receives a Markdown file path through command-line arguments and outputs the extracted content according to the selected mode.

Project information

****Author:**** danya_purple02

****Creation date:**** 05.03.2026

****Repository on GitHub:**** [TP-1](<https://github.com/danya-purple02/TP-1.git>)

Description

This project was created as part of a laboratory work on C++ programming.

The application works with Markdown documents and can extract:

- headers;
- paragraphs;
- ordered one-level lists.

Command-line arguments are processed using the CLI11 library.

Project structure

```
```text
MarkdownParser/
|
├─ .vscode/
| └─ tasks.json
|
├─ build/
| └─ main.exe
|
├─ data/
| └─ test Markdown files
|
├─ src/
| └─ main.cpp
|
├─ third_party/
| └─ CLI11/
```

```
|
|─ README.md
|─ README_RU.md
|─ README_EN.md
|
|
```

## ## Requirements

To build and run the project, you need:

- Windows;
- Visual Studio Code;
- C/C++ extension for VS Code;
- MinGW-w64 / WinLibs GCC;
- CLI11 library.

The project uses the following compiler:

```
```text
G++ 16.1.0
```
```

## ## Build

The project is built through the configured VS Code build task.

Press:

```
```text
Ctrl + Shift + B
```
```

After the build, the executable file will be created in the following directory:

```
```text
build/main.exe
```
```

Manual build command example:

```
```powershell
C:\drivers\mingw64\bin\g++.exe -fdiagnostics-color=always -g -std=c++20 -I
third_party\CLI11\include src\main.cpp -o build\main.exe
```
```

## ## Run

Example launch command:

```
```powershell
.\build\main.exe --mode all --file ".\data\markdown_mixed.md"
```

```
```
```

## ## Command-line arguments

```
```text
--mode    parsing mode
--file    path to the Markdown file
```
```

Example:

```
```powershell
.\build\main.exe --mode all --file ".\data\test.md"
```
```

## ## Used libraries

### ### CLI11

This project uses the [CLI11](<https://github.com/CLIUtils/CLI11>) library for command-line argument parsing.

CLI11 is a header-only library, so it is enough to download its source files into the project and add the include directory to the build configuration. The project uses the following include directive:

```
```cpp
#include <CLI/CLI.hpp>
```
```

To install CLI11 into the project, run the following command from the repository root directory:

```
```powershell
git clone https://github.com/CLIUtils/CLI11.git third_party\CLI11
```
```

After running the command, the following file should appear in the project:

```
```text
third_party\CLI11\include\CLI\CLI.hpp
```
```

## ## Notes

The project is configured to use UTF-8 encoding for correct processing and output of Russian text in the terminal.

## Приложение 6 – содержимое файла “*README\_RU.md*”

Этот README сгенерирован при помощи искусственного интеллекта.

### # MarkdownParser

Консольное приложение на C++ для разбора Markdown-файлов и извлечения выбранных элементов разметки.

Программа получает путь к Markdown-файлу через аргументы командной строки и выводит найденные элементы в соответствии с выбранным режимом работы.

### ## Информация о проекте

**\*\*Автор:\*\*** danya\_purple02

**\*\*Дата создания:\*\*** 05.03.2026

**\*\*Репозиторий на GitHub:\*\*** [TP-1](<https://github.com/danya-purple02/TP-1.git>)

### ## Описание

Проект создан в рамках лабораторной работы по программированию на C++.

Приложение работает с Markdown-документами и умеет извлекать:

- заголовки;
- абзацы;
- упорядоченные одноуровневые списки.

Для обработки аргументов командной строки используется библиотека CLI11.

### ## Структура проекта

```
```text
MarkdownParser/
|
├─ .vscode/
|   └─ tasks.json
|
├─ build/
|   └─ main.exe
|
├─ data/
|   └─ тестовые Markdown-файлы
|
├─ src/
|   └─ main.cpp
|
├─ third_party/
|   └─ CLI11/
```

```
|
|─ README.md
|─ README_RU.md
|─ README_EN.md
|
|
|
```

Требования

Для сборки и запуска проекта нужны:

- Windows;
- Visual Studio Code;
- расширение C/C++ для VS Code;
- MinGW-w64 / WinLibs GCC;
- библиотека CLI11.

Используемый компилятор:

```
```text
G++ 16.1.0
```
```

Сборка

Проект собирается через задачу VS Code.

Нажмите:

```
```text
Ctrl + Shift + B
```
```

После сборки исполняемый файл будет создан в папке:

```
```text
build/main.exe
```
```

Пример ручной команды сборки:

```
```powershell
C:\drivers\mingw64\bin\g++.exe -fdiagnostics-color=always -g -std=c++20 -I
third_party\CLI11\include src\main.cpp -o build\main.exe
```
```

Запуск

Пример запуска:

```
```powershell
.\build\main.exe --mode all --file ".\data\markdown_mixed.md"
```

```

Аргументы командной строки

```
```text
--mode режим разбора файла
--file путь к Markdown-файлу
```
```

Пример:

```
```powershell
.\build\main.exe --mode all --file ".\data\test.md"
```
```

Используемые библиотеки

CLI11

В проекте используется библиотека [CLI11](<https://github.com/CLIUtils/CLI11>) для обработки аргументов командной строки.

CLI11 является header-only библиотекой, поэтому для её использования достаточно скачать исходные файлы в проект и указать путь к заголовочным файлам при сборке. В проекте используется подключение:

```
```cpp
#include <CLI/CLI.hpp>
```
```

Для установки CLI11 в проект выполните команду из корневой папки репозитория:

```
```powershell
git clone https://github.com/CLIUtils/CLI11.git third_party\CLI11
```
```

После выполнения команды в проекте должен появиться файл:

```
```text
third_party\CLI11\include\CLI\CLI.hpp
```
```

Примечания

Проект настроен на использование UTF-8 для корректной обработки и вывода русского текста в терминале.

Приложение 7 – содержимое файла “*README.md*”

This README generated with the assistance of AI.

MarkdownParser

Language / Язык

- [English version](README_EN.md)
- [Русская версия](README_RU.md)

****Author:**** danya_purple02

****Creation date:**** 05.03.2026

****Repository:**** [TP-1](https://github.com/danya-purple02/TP-1.git)
